

Abdul Rauf Babar | 23L0804 | BCS-7A

<https://github.com/rauf-babar/cs3006-a2>

1. Machine Declaration

All measurements for this assignment were taken on the following machine:

Laptop	HP Victus Gaming Laptop 15-fa2
CPU	13th Gen Intel Core i5-13420H
Physical cores, C	8
Hardware threads, T	12
SIMD support	AVX2
SIMD width, W	8 single-precision lanes
Maximum CPU frequency	4.6 GHz
RAM	16 GB DDR4-3200
Operating System	Ubuntu 26.04.1 LTS

Kernel	Linux 7.0.0-30-generic
Compiler	GCC/G++ 15.2.0
ISPC version	1.31.0

The laptop was plugged in and the CPU governor was changed from powersave to performance.

Each reported configuration was run at least five times. The minimum runtime was used as the reported result. I also recorded the minimum and maximum timings for at least one configuration to show the amount of measurement variation. Background applications were closed during measurements, and the machine was allowed to cool between long runs when necessary.

2. Build Notes

Two small include fixes were needed to compile the Stanford starter code with GCC 15.2.0.

In: prog1_mandelbrot_threads/main.cpp

I added: `#include <cstring>`

In: prog1_mandelbrot_threads/mandelbrotThread.cpp

I added: `#include <cstdlib>`

These changes only fixed compilation issues and did not change the behavior of the program.

The ISPC compiler used for the assignment was:

Intel ISPC 1.31.0

The ISPC programs were compiled for AVX2 where required.

3. Program 1

3.1 Initial Decomposition

My first implementation divided the image into continuous horizontal blocks of rows.

For N threads, each thread received approximately height / N rows.

The starting row of a thread was based on:

$$\text{startRow} = \text{threadId} \times \text{rowsPerThread}$$

The last thread handled any remaining rows.

For example, with two threads, one thread processed the upper half of the image and the other processed the lower half.

The output was correct, but increasing the number of threads did not give linear speedup.

3.2 Speedup of the Initial Decomposition

The following table shows the View 1 measurements for the original block decomposition.

Threads	Minimum Parallel Time (ms)	Speedup
2	143.011	1.99×
3	174.184	1.63×
4	116.875	2.44×
5	115.613	2.46×
6	88.327	3.22×
7	84.450	3.37×
8	71.152	4.00×

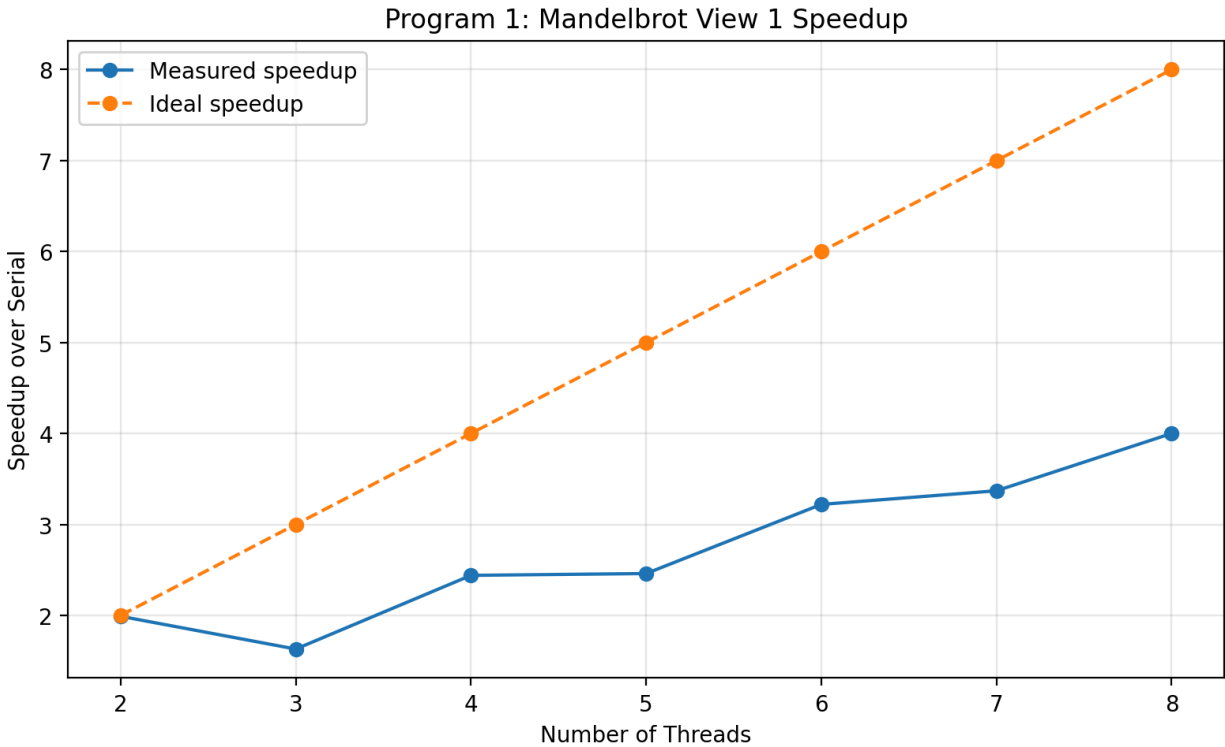


Figure 1: Speedup of the initial contiguous-row decomposition for Mandelbrot View 1.

The scaling was not linear. For example, 3 threads gave only 1.63× speedup, while 8 threads gave about 4.00× instead of the ideal 8×.

The unusual 3-thread result also shows that simply increasing the thread count does not guarantee better performance when the work is not divided evenly.

3.3 Per-Thread Timing of the Initial Decomposition

One representative 8-thread View 1 run gave:

Thread	Time (ms)
0	5.350
1	33.319
2	54.218
3	71.296

4	71.539
5	47.658
6	33.585
7	5.402

The fastest thread took about: 5.35 ms

while the slowest took about: 71.54 ms

The ratio was: $71.54 / 5.35 \approx 13.4$

This was a large load imbalance. The reason is that all Mandelbrot rows do not require the same amount of work. Some pixels escape after only a few iterations, while pixels near or inside the Mandelbrot set require many more iterations.

Because the original method gave each thread a continuous part of the image, some threads received much more expensive regions than others.

Threads 0 and 7 finished very quickly, while threads around the middle of the image continued working much longer. The whole program still had to wait for the slowest thread to finish.

3.4 Improved Decomposition

I changed the work distribution from continuous row blocks to interleaved rows.

With N threads, thread t processes:

$t, t + N, t + 2N, t + 3N, \dots$

For example, with eight threads:

- Thread 0 processes rows 0, 8, 16, 24, ...
- Thread 1 processes rows 1, 9, 17, 25, ...
- Thread 2 processes rows 2, 10, 18, 26, ...
- Thread 3 processes rows 3, 11, 19, 27, ...
- and so on.

This spreads different parts of the image across the threads instead of giving each thread one continuous region.

The solution is still static. It does not use a shared work queue, locks, or any extra synchronization.

3.5 Per-Thread Timing After Interleaving

A representative 8-thread View 1 run with the new decomposition gave:

Thread	Time (ms)
0	40.470
1	47.691
2	52.773
3	36.625
4	36.482
5	36.623
6	51.685
7	42.407

The fastest thread took about: 36 . 48 ms

and the slowest took about: 52 . 77 ms

The ratio was: $52.77 / 36.48 \approx 1.45$

The original block decomposition had a ratio of about **13.4**, while the interleaved version reduced it to about **1.45**.

This shows that the work was distributed much more evenly after changing the decomposition.

3.6 Final Performance Results

The improved implementation was tested on both Mandelbrot views using:

- 8 threads
- T = 12 threads

- $2T = 24$ threads

View 1

Threads	Minimum Parallel Time (ms)	Best Speedup
8	50.048	5.70×
12	33.796	8.42×
24	32.635	8.72×

View 2

Threads	Minimum Parallel Time (ms)	Best Speedup
8	29.691	5.40×
12	20.761	7.70×
24	20.056	7.96×

For View 1, the original 8-thread block decomposition reached about: 4.00×

After changing to interleaved rows, the 8-thread result increased to: 5.70×

The relative increase in speedup was: $5.70 / 4.00 = 1.425$

which is about a **42.5% improvement** in achieved speedup.

3.7 Result at T Hardware Threads

My machine has:

$T = 12$ hardware threads

The ideal speedup with 12 threads would be: 12×

The measured results were:

- **View 1:** 8.42×
- **View 2:** 7.70×

For View 1: $8.42 / 12 \times 100 \approx 70.2\%$

For View 2: $7.70 / 12 \times 100 \approx 64.2\%$

So the program reached about **70.2% of ideal scaling on View 1** and **64.2% on View 2**.

The result is below ideal because hardware threads do not provide the same amount of additional execution capacity as separate physical cores. There is also thread scheduling overhead, shared CPU resources, cache effects, and some remaining difference in workload between threads.

3.8 2T Thread Result

The required oversubscribed configuration for my machine is:

2T = 24 threads

The best measured results were:

- **View 1:** 8.72×
- **View 2:** 7.96×

Comparison:

View	12 Threads	24 Threads
View 1	8.42×	8.72×
View 2	7.70×	7.96×

For View 1: $8.72 / 8.42 \approx 1.036$

This is only about a **3.6% improvement**.

For View 2: $7.96 / 7.70 \approx 1.034$

This is about a **3.4% improvement**.

Using 24 software threads therefore gave only a small improvement over 12 threads.

The machine only provides 12 hardware threads, so 24 software threads have to share the same hardware execution resources. Adding more software threads does not create more physical processing resources and can also increase scheduling and resource contention.

3.9 Measurement Spread

For the original 8-thread View 1 block decomposition, the five parallel runtimes were:

71.241 ms, 71.367 ms, 71.300 ms, 71.152 ms, 71.172 ms

Therefore:

- **Minimum:** 71.152 ms
- **Maximum:** 71.367 ms
- **Spread:** 0.215 ms

The percentage spread was: $0.215 / 71.152 \times 100 \approx 0.30\%$

This small variation shows that the final measurements were stable after changing the CPU governor to performance.

3.10 Program 1 Summary

The first contiguous-row decomposition produced correct output but had poor load balance. In the representative 8-thread timing test, the fastest thread took about 5.35 ms while the slowest took about 71.54 ms.

Changing the decomposition to interleaved rows distributed the work more evenly. The fastest-to-slowest ratio dropped from about 13.4 to about 1.45.

With the final implementation, the program achieved:

- **8 threads:** 5.70× on View 1 and 5.40× on View 2
- **12 threads:** 8.42× on View 1 and 7.70× on View 2
- **24 threads:** 8.72× on View 1 and 7.96× on View 2

The 24-thread case gave only a small improvement over 12 threads because the machine has only 12 hardware threads.

4. Program 2

The vectorized implementation processes VECTOR_WIDTH elements at a time. Masks are used for two purposes:

- to handle the final partial vector when N is not divisible by VECTOR_WIDTH;
- to disable lanes whose exponent calculation has already finished.

The implementation was verified against the serial version and passed the correctness test for every vector width tested.

4.1 Vector Utilization

Vector Width	Vector Utilization	Correctness
2	90.3%	Passed
4	85.9%	Passed
8	80.5%	Passed
16	81.4%	Passed

The highest utilization was obtained with vector width 2 at **90.3%**.

As the vector width increased, utilization generally decreased:

$$90.3\% \rightarrow 85.9\% \rightarrow 80.5\%$$

from widths 2 to 8.

The reason is that the exponent values are different for different elements. Some lanes finish their exponentiation loop earlier than others. Once a lane has finished, its mask becomes inactive, but the remaining lanes must continue executing until the element with the largest remaining exponent is finished.

With a small vector width, fewer elements are grouped together, so there is less chance of a large difference in the amount of work performed by the lanes of the same vector.

With wider vectors, more elements execute together. If only a few lanes still have exponent iterations remaining, the vector instruction still executes while the completed lanes remain masked off. These inactive lanes reduce vector utilization.

The width-16 result was **81.4%**, which is slightly higher than the width-8 result of **80.5%**. Therefore, the measured values are not perfectly monotonic. This small increase is caused by

the particular distribution of exponent values in the input. However, both width 8 and width 16 still have lower utilization than widths 2 and 4.

Overall, the results show that wider vectors can expose more unused lanes when different elements require different numbers of loop iterations. This is an example of SIMD divergence caused by data-dependent control flow.

4.2 Correctness

The vector implementation passed the result verification at all tested widths:

- Width 2 — Passed
- Width 4 — Passed
- Width 8 — Passed
- Width 16 — Passed

For the default vector width of 4, the program reported:

- **Total Vector Instructions:** 144
- **Vector Utilization:** 85.9%
- **Utilized Vector Lanes:** 495
- **Total Vector Lanes:** 576

Therefore: $495 / 576 \times 100 \approx 85.9\%$

which matches the utilization reported by the provided vector-intrinsics framework.

5. Program 3

5.1 Part 1: ISPC SIMD

The ISPC compiler was configured for AVX2 with: $W = 8$

The maximum expected SIMD speedup is therefore: $8\times$

Final measurements:

View	Minimum Serial Time (ms)	Minimum ISPC Time (ms)	Speedup	Fraction of W
1	142.902	40.340	3.54×	44.3%

View	Minimum Serial Time (ms)	Minimum ISPC Time (ms)	Speedup	Fraction of W
2	80.345	27.966	2.87×	35.9%

The measured speedup is below the ideal 8× because different Mandelbrot pixels require different numbers of iterations.

ISPC executes a group of program instances using SIMD lanes. If some pixels finish early while other pixels in the same group continue iterating, the finished lanes become inactive and cannot contribute useful work until the group completes.

This effect is stronger in View 2, which is consistent with its lower measured speedup of 2.87× compared with 3.54× for View 1.

5.2 Part 2: ISPC Tasks

The starter implementation used two tasks.

For View 1, the five-run 2-task measurements gave a minimum task time of: 20.221 ms

Using the minimum serial time from those runs: $142.351 / 20.221 \approx 7.04\times$

So the 2-task version achieved approximately: 7.04× speedup over serial

The minimum non-task ISPC time was: 40.205 ms

Therefore, the 2-task version was approximately: $40.205 / 20.221 \approx 1.99\times$

faster than the single-core ISPC implementation.

Task Count Selection

I tested several task counts and compared their performance on both views.

Tasks	Best Observed View 1 Speedup	Best Observed View 2 Speedup
2	7.13×	4.84×
8	13.96×	11.86×
16	19.21×	15.09×

Tasks	Best Observed View 1 Speedup	Best Observed View 2 Speedup
32	27.14×	19.14×
40	29.02×	18.97×
80	31.45×	22.99×
100	30.68×	21.66×

I selected: 80 tasks

With an image height of 800: $800 / 80 = 10$ rows per task

Using more tasks gives the runtime more independent pieces of work to distribute across the cores. This helps reduce the effect of uneven Mandelbrot computation across different image regions.

The performance improved up to 80 tasks. Increasing the count to 100 reduced performance on both views, which indicates that the additional task-management overhead was no longer compensated by better load balancing.

The final implementation therefore uses: `rowsPerTask = height / 80`

And: `launch[80]`

5.3 Final Task Results

For my machine: $C = 8$

And: $W = 8$

Therefore: $C \times W = 64$

The theoretical combined multicore and SIMD ceiling is: $64\times$

Final five-run measurements with 80 tasks:

View	Minimum Serial Time (ms)	Minimum Task Time (ms)	Speedup	Fraction of $C \times W$
1	142.902	4.666	30.63×	47.9%

View	Minimum Serial Time (ms)	Minimum Task Time (ms)	Speedup	Fraction of $C \times W$
2	80.345	3.498	22.97×	35.9%

For View 1: $30.63 / 64 \times 100 \approx 47.9\%$

For View 2: $22.97 / 64 \times 100 \approx 35.9\%$

The final task implementation was also faster than the ISPC-only implementation.

For View 1: $40.340 / 4.666 \approx 8.65\times$

For View 2: $27.966 / 3.498 \approx 7.99\times$

The result remains below the $C \times W = 64$ ceiling because the two forms of parallelism are not perfectly utilized at the same time. SIMD lanes still suffer from different Mandelbrot iteration counts, tasks do not contain exactly equal computational work, and task scheduling introduces overhead.

The final measured speedups were:

View 1: 30.63×

View 2: 22.97×

6. Program 4

For this machine: $W = 8$ $C = 8$

Maximum-Speedup Input

The maximum-speedup input used the same value for every element:

`values[i] = 2.999f;`

Measured minimum times:

Implementation	Time
Serial	1194.338 ms

Implementation	Time
ISPC	261.142 ms

ISPC speedup: $1194.338 / 261.142 = 4.57\times$

Relative to the SIMD width: $4.57 / 8 \times 100 = 57.1\%$ of W

This input gives good SIMD utilization because all eight lanes receive the same value and therefore require the same number of Newton iterations. The lanes progress through the loop together, so very little work is lost to inactive lanes.

Using a value such as 2.999f also gives each element enough iterative work for SIMD execution to be useful. Using 1.0f for every element would make every lane finish immediately, but the computation would be too small for that to be a useful maximum-speedup case.

Minimum-Speedup Input

The minimum-speedup input was:

```

if (i % 8 == 0)
    values[i] = 2.999f;
else
    values[i] = 1.0f;

```

For each group of eight consecutive elements, one element requires several Newton iterations while the other seven require no loop iterations.

Measured minimum times:

Implementation	Time
Serial	168.830 ms
ISPC	261.037 ms

ISPC speedup: $168.830 / 261.037 = 0.65\times$

Relative to the SIMD width: $0.65 / 8 \times 100 \approx 8.1\%$ of W

The input causes severe SIMD divergence. With $W = 8$, seven lanes finish immediately because their value is 1.0f, while one lane continues executing the Newton iteration for 2.999f.

The SIMD group cannot finish until that remaining lane completes. During most of the loop, only one of the eight lanes is doing useful work while the other seven are inactive.

The serial implementation does not suffer from this lane-divergence problem. It quickly processes the seven easy values and performs the expensive loop only for every eighth element. As a result, the ISPC version is slower than serial for this input, producing a speedup of only 0.65×

Comparison

Input	Serial Time	ISPC Time	ISPC Speedup
Maximum-speedup	1194.338 ms	261.142 ms	4.57×
Minimum-speedup	168.830 ms	261.037 ms	0.65×

The difference comes from SIMD lane utilization. The maximum-speedup input keeps all lanes doing approximately equal work, while the minimum-speedup input leaves only one lane active during most of the iterative computation.

7. Program 5

The best measured times from five runs were:

Implementation	Minimum Time	Reported Bandwidth	GFLOPS
ISPC	14.818 ms	20.113 GB/s	2.699
ISPC with tasks	13.167 ms	22.635 GB/s	3.038

The speedup from using tasks was: $14.818 / 13.167 = 1.13\times$

The small speedup occurs because SAXPY performs very little computation for each element. Each result requires only one multiplication and one addition, while large arrays must continually

be read from and written to memory. As more cores execute SAXPY, they share the same memory subsystem, so adding parallel tasks cannot provide near-linear speedup once memory bandwidth becomes the main limitation.

Therefore, I would not expect a rewrite using more CPU threads to produce near-linear multicore speedup. The computation is already simple and regular, and performance is limited mainly by the rate at which data can be transferred to and from memory rather than by available arithmetic execution resources.

Memory Bandwidth Comparison

The machine has one installed 16 GB DDR4-3200 SODIMM, with the second memory slot empty. The active memory configuration is therefore single-channel.

For one 64-bit DDR4-3200 channel: $3200 \text{ MT/s} \times 8 \text{ bytes} = 25.6 \text{ GB/s}$

So the theoretical peak memory bandwidth is: 25.6 GB/s

The best task implementation reported: 22.635 GB/s

The program calculates this value using 1024^3 bytes per GB, so the printed value is numerically equivalent to approximately: 24.304 GB/s in decimal bandwidth units.

Compared with the theoretical DDR4-3200 peak: $24.304 / 25.6 \times 100 \approx 94.9\%$

Thus, the best task implementation reaches approximately 94.9% of the nominal single-channel memory bandwidth. This supports the observation that SAXPY is primarily memory-bandwidth limited. Since the memory subsystem is already being used close to its theoretical peak, adding more parallel computation provides only a small additional speedup.

Measurement Spread

For the ISPC-with-tasks configuration, the five measured runtimes ranged from 13.167 ms to 15.473 ms, giving a spread of 2.306 ms.

8. Program 6

8.1 Dataset Verification

The full dataset contained: $M = 1,000,000$

The dataset was verified before taking the final measurements using:

```
md5sum data.dat
```

The resulting checksum was: 3a25f24193f4fdca82ee4cb2737fd5bb

This matches the checksum specified for the full CS3006 dataset.

8.2 Baseline and Profiling

The original implementation was profiled using `CycleTimer`. I measured the three main phases of every K-means iteration:

- `computeAssignments`
- `computeCentroids`
- `computeCost`

The original implementation was run five times using the full dataset.

Run	Total Time
1	28224.695 ms
2	27767.594 ms
3	26769.884 ms
4	26744.287 ms
5	26721.861 ms

Following the measurement protocol, the minimum runtime was used:

Baseline minimum = 26721.861 ms

All five runs completed in:

101 iterations

The phase timings from the fastest baseline run were:

Phase	Time	Fraction
computeAssignments	17155.481 ms	64.20%
computeCentroids	4326.607 ms	16.19%
computeCost	5239.728 ms	19.61%

The dominant phase was therefore computeAssignments.

The measured hotspot fraction was: $f = 0.6420$

The baseline measurements ranged from: 26721.861 ms to 28224.695 ms

Therefore, the baseline measurement spread was: $28224.695 - 26721.861 = 1502.834$ ms

8.3 Hardware-Relative Amdahl Target

My machine has: $T = 12$ hardware threads

Using the measured hotspot fraction: $f = 0.6420$

the Amdahl-law ceiling for parallelizing this hotspot is:

$$S_{\max} = 1 / ((1 - f) + f / T)$$

Substituting the measured values:

$$S_{\max} = 1 / ((1 - 0.6420) + 0.6420 / 12)$$

$$S_{\max} \approx 2.430\times$$

The CS3006 target is 80% of this ceiling: $0.80 \times 2.430 \approx 1.944\times$

Therefore, the hardware-relative performance target for my machine was approximately: $1.944\times$

8.4 First Optimization: Parallelizing computeAssignments

The first optimization targeted `computeAssignments`, since it accounted for approximately 64.20% of the baseline runtime.

Instead of dividing the computation by cluster, I divided the M data points among the 12 worker threads.

Each thread received a separate range of data points and, for every point in its range, compared that point with all K cluster centroids.

This decomposition avoids write conflicts because each data point has exactly one corresponding `clusterAssignments[m]` entry, and only one thread writes to that entry.

The assignment-parallel implementation was also run five times.

Run	Total Time
1	14720.730 ms
2	13649.409 ms
3	13663.364 ms
4	13671.855 ms
5	13726.426 ms

The minimum runtime was: 13649.409 ms

The phase timings from the fastest run were:

Phase	Time	Fraction
computeAssignments	3909.666 ms	28.64%
computeCentroids	4434.433 ms	32.49%

computeCost	5305.271 ms	38.87%
-------------	-------------	--------

The speedup after the first optimization was:

$$26721.861 / 13649.409 \approx 1.958\times$$

The iteration count remained: 101

This result exceeded the hardware-relative target of approximately:
1.944×

The profiling result also showed that the bottleneck had changed. computeAssignments fell from 64.20% of the runtime to 28.64%, while computeCost became the largest remaining phase at: 38.87%

This measurement led to the second optimization.

8.5 Second Optimization: Parallelizing computeCost

The next optimization targeted computeCost.

The data points were again divided among 12 threads.

A direct shared accumulation into the same cluster-cost values would introduce data races because several threads could update the same cluster simultaneously.

To avoid this, each thread was given its own private K-element cost array.

Each worker accumulated the costs for its assigned data-point range into its private array.

After all threads completed, the main thread combined the private arrays into the final cluster costs.

This allowed the cost computation to execute in parallel without concurrent writes to the same accumulation variables.

After this optimization, I performed the final five-run measurement pass.

8.6 Final Measurements

The final implementation was run five times on the complete dataset.

Run	Total Time
1	12749.505 ms
2	12109.583 ms
3	12216.737 ms
4	12162.234 ms
5	11881.468 ms

Following the measurement protocol, the minimum runtime was used:

Final minimum = 11881.468 ms

The final speedup over the five-run baseline minimum was:

$26721.861 / 11881.468 \approx 2.249\times$

Therefore, the final implementation achieved approximately:

2.25× speedup

The final measurements ranged from:

11881.468 ms to 12749.505 ms

The spread was:

$12749.505 - 11881.468 = 868.037$ ms

All final runs completed in:

101 iterations

so the optimized implementation preserved the convergence behavior of the original implementation.

8.7 Comparison with the Hardware-Relative Target

The measured Amdahl ceiling based on the original computeAssignments hotspot was:

$S_{\max} \approx 2.430\times$

The required CS3006 target was:

$0.80 \times S_{\max} \approx 1.944\times$

The final measured speedup was:

2.249×

Relative to the computeAssignments-based Amdahl ceiling:

$2.249 / 2.430 \times 100 \approx 92.55\%$

Therefore, the final implementation achieved approximately:

92.55% of the computeAssignments-based Amdahl ceiling

used to establish the hardware-relative CS3006 target.

The result is comfortably above the required:

1.944×

target.

8.8 Comparison with the Stanford Result

The Stanford reference speedup for Program 6 is:

2.1×

My final measured speedup was:

2.249×

Therefore, my result was approximately:

$2.249 - 2.1 = 0.149\times$

above the Stanford reference.

In percentage terms:

$(2.249 / 2.1 - 1) \times 100 \approx 7.1\%$

So the final implementation achieved a speedup about:

7.1% higher than the Stanford 2.1× reference

on my declared machine.

8.9 Final Profiling

For the fastest final run, the measured phase times were:

Phase	Time	Fraction
computeAssignments	3868.521 ms	32.56%
computeCentroids	4420.607 ms	37.21%
computeCost	3592.299 ms	30.23%

Compared with the baseline:

Phase	Baseline Fraction	Final Fraction
computeAssignments	64.20%	32.56%
computeCentroids	16.19%	37.21%
computeCost	19.61%	30.23%

Initially, computeAssignments dominated the runtime at 64.20%.

After parallelizing both computeAssignments and computeCost, no single phase dominated the execution to the same extent. The three phases were much more evenly distributed in the final implementation.

8.10 K-Means Plots

Initial K-Means State

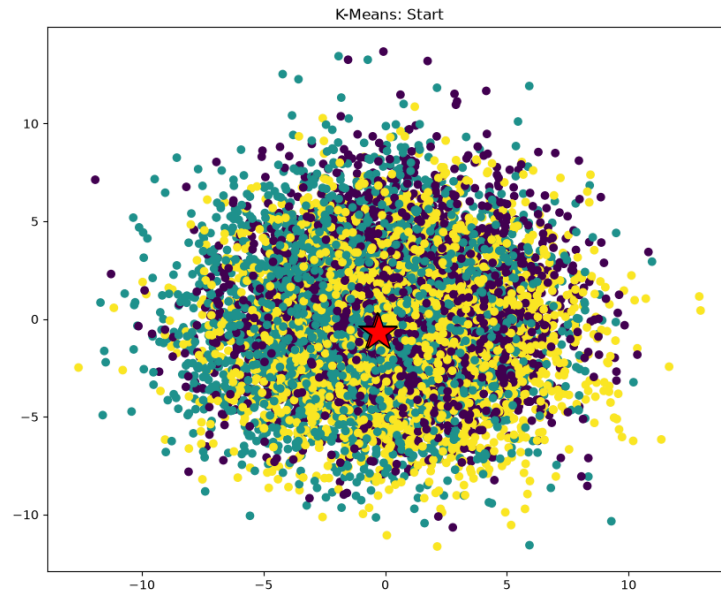


Figure: Initial cluster assignments and centroid positions before K-means convergence.

start.png shows the initial state of the generated dataset before the iterative K-means process converges.

Final K-Means State

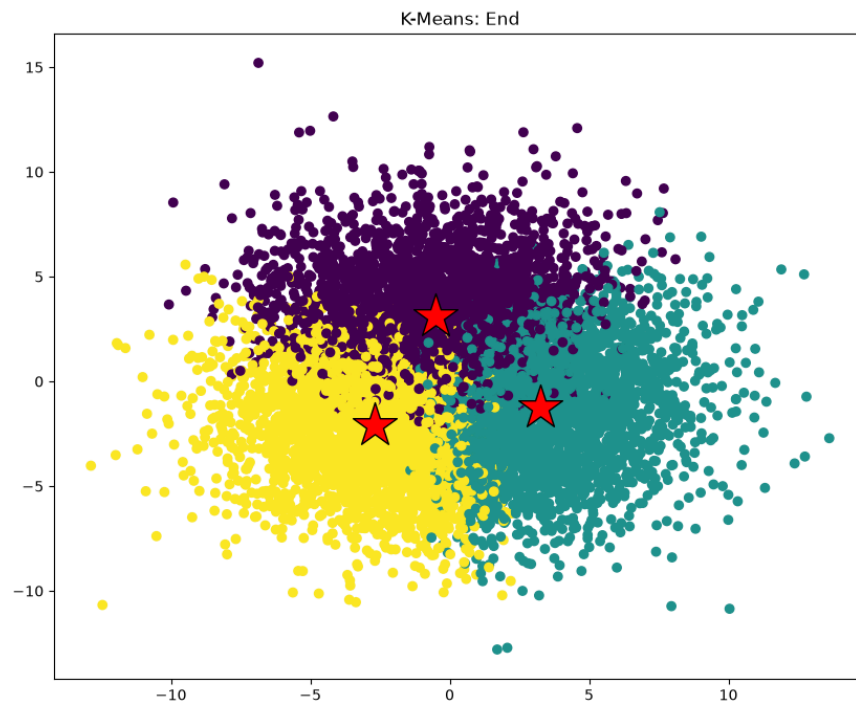


Figure: Final cluster assignments and centroid positions after K-means convergence.

end .png shows the state after the K-means algorithm has converged.

The final plot shows three coherent clusters, with the centroid markers located within the corresponding cluster regions.

The original data contains 100 dimensions, while the plots show only two dimensions obtained using PCA. Therefore, an individual point may appear visually closer to another centroid in the two-dimensional plot even though its assignment is correct in the original 100-dimensional space.

The plots provide a visual correctness check that the optimized parallel implementation still produces coherent clustering after the performance changes.

8.11 Program 6 Summary

The original implementation took a minimum of:

26721.861 ms

Profiling showed that `computeAssignments` accounted for approximately: 64.20% of the runtime.

Parallelizing `computeAssignments` reduced the runtime to:
13649.409 ms

for a speedup of approximately: 1.958×

Profiling after that change showed that `computeCost` had become the largest remaining phase.

I then parallelized `computeCost` using private per-thread cost arrays followed by a reduction.

The final implementation achieved a minimum runtime of: 11881.468 ms

and an overall speedup of: 2.249×

This exceeded the hardware-relative CS3006 target of approximately:
1.944×

and was also above the Stanford reference result of: 2.1×

The optimized implementation continued to converge in:

101 iterations

and the final plots showed coherent clustering.

9. Bonus Programs

9.1. Program 3 Extra Credit: Threads vs ISPC Tasks

A C++ thread and an ISPC task both allow independent work to execute in parallel, but they represent different levels of abstraction.

With `std::thread`, each created thread is an operating-system thread. The programmer explicitly creates the thread and later waits for it using `join()`. Creating a thread therefore involves operating-system resources such as a thread stack and scheduling information. If there are more threads than available hardware threads, the operating system must repeatedly schedule and context-switch between them.

ISPC tasks are lighter-weight units of work. A launch statement creates tasks that are submitted to the ISPC task system. The runtime uses a relatively small pool of worker threads to execute these tasks. The number of tasks can therefore be much larger than the number of physical cores or hardware threads without requiring one operating-system thread per task.

For example, launching 10,000 C++ threads would attempt to create 10,000 operating-system threads. Only a small number could execute simultaneously on the available CPU hardware threads. The remaining threads would wait to be scheduled, while memory consumption, thread-creation cost, scheduling overhead, and context switching would become significant. Depending on the system, creating that many threads may also encounter operating-system resource limits.

Launching 10,000 ISPC tasks is different. The runtime does not normally create 10,000 operating-system threads. Instead, the tasks form a collection of work that is processed by the existing worker threads. As a worker finishes one task, it can take another. This makes it practical to create many more tasks than there are CPU cores and can also improve load balancing when different tasks require different amounts of work.

Another difference is how work is expressed. With threads, the programmer is explicitly managing concurrent execution using thread creation and synchronization. With ISPC tasks, the programmer describes pieces of work that are safe to execute independently, while the task runtime is responsible for mapping those tasks onto the available CPU threads.

Therefore, a large number of ISPC tasks can be useful for exposing parallelism and improving load balancing, whereas creating the same number of operating-system threads would usually introduce excessive overhead. Tasks are units of work scheduled

onto threads; they are not equivalent to creating a new operating-system thread for every task.

9.2. Program 5 Extra Credit: Why SAXPY Transfers 4N Floats

SAXPY performs:

$$\text{result}[i] = \text{scale} * X[i] + Y[i]$$

At first, this appears to require three memory transfers per element:

1. Read $X[i]$
2. Read $Y[i]$
3. Write $\text{result}[i]$

However, normal cached stores on the CPU use a write-allocate policy. When the processor writes to a cache line of result that is not already present in the cache, it first brings that cache line into the cache. This is commonly called a Read For Ownership (RFO).

Therefore, the effective memory traffic is:

$$X \text{ read} + Y \text{ read} + \text{result RFO} + \text{result write-back}$$

or:

$$N + N + N + N = 4N \text{ floats}$$

Thus the total bandwidth is modeled as:

$$\text{TOTAL_BYTES} = 4 * N * \text{sizeof(float)}$$

The extra $N * \text{sizeof(float)}$ comes from loading the result cache lines before modifying them. After modification, the dirty cache lines must eventually be written back to memory.

This is why the bandwidth calculation uses four rather than three memory transfers per element, even though the SAXPY expression itself explicitly reads only two arrays and writes one.

9.3. Program 2 Extra Credit: Vectorized Array Sum

I implemented `arraySumVector()` using the provided CS149 vector intrinsics.

The input is processed `VECTOR_WIDTH` elements at a time. Each loaded vector is added to a vector accumulator. After all input elements have been processed, the accumulator is stored into a temporary array and its `VECTOR_WIDTH` partial sums are added together to produce the final result.

This reduces the main accumulation work from approximately N scalar additions to:

$N / \text{VECTOR_WIDTH} + \text{VECTOR_WIDTH}$ operations.

The implementation was tested with multiple input sizes and produced the expected result for the assignment-scale tests.

The implementation is located in `prog2_vecintrin/main.cpp` in the function `arraySumVector()`.

10. AI Use Disclosure

ChatGPT was used to help understand assignment instructions and technical concepts, interpret the meaning of different steps and program outputs, assist with implementation and debugging guidance, and improve the clarity and wording of the write-up. All program executions and reported measurements were performed on my own declared machine.